PROVEIT / DIOPHANTINE CONSISTENCY

Four Further Lean Proof Simplifications

Polynomial identities, list sums,
assignment restriction, and exponent divisibility

Prepared for Vladimir Reshetnikov

26 October 2026

Main result. A checked transport theorem lets equality of integer polynomials justify equality-of-splits semantics over every additively cancellative commutative semiring. It supports arbitrary internal naturals without identifying them with external natural numbers.

Concrete companions. A complete sparse-coefficient module refactor removes four of five list inductions and ten source lines. A smaller exponent proof removes four lines. An environment proof proposes twelve fewer lines; its abstract mechanism is checked, while its full object-model integration remains uncompiled.

Exclusion boundary. Earlier proposals are treated as already applied, including optional and uncompiled ones. The audit covers 23 placed packets and five incoming ZIPs. A late packet caused a coprime-divisibility candidate to be withdrawn.

Source and compiler baseline: `89226e3921bc`.
Closing novelty review: `7df43cdfacb4`.
Lean 4.32.0, with the repository's pinned Mathlib.
Source, patches, compiler receipts, and exclusion evidence are bundled.
No whole-project build or global source-size reduction is claimed.

Contents

1 Findings and recommended order **2**
1.1 Source baseline and closing refresh . . . 2

2 What has already been proposed **3**
2.1 Nearest-precedent comparison . . . 3

3 P1: transport polynomial identities into internal arithmetic **4**
3.1 The proof obligation left by a semiring evaluator . . . 4
3.2 Proof by a natural cross-sum identity . . . 4
3.3 Checked Lean interface . . . 5
3.4 Concrete client: the conditional selection row . . . 5
3.5 The hypotheses are substantive . . . 6
3.6 Adoption cost and limits . . . 6

4 P2: give sparse-list expansion one monomial-sum view **6**
4.1 The repeated reasoning . . . 6
4.2 The new proof structure . . . 7
4.3 Why the executable behavior stays exact . . . 7
4.4 Measurements and checking scope . . . 7

5 P3: transport quantification by restriction surjectivity **8**
5.1 The actual environment-extension problem . . . 8
5.2 Use the existing extension principle . . . 8
5.3 Boundary cases and distinctness . . . 9
5.4 Evidence and outstanding integration . . . 9

6 P4: use the order obstruction on a residual exponent **9**
6.1 A smaller proof of the existing theorem . . . 9
6.2 Optional arithmetic API: recover a block power directly . . . 10
6.3 Necessary guards . . . 10

7 Validation, costs, and adoption gates **11**
7.1 Before adoption . . . 11

8 Archive guide and reproduction **12**

Primary sources **12**

1 Findings and recommended order

The remaining opportunities are narrower than the earlier broad refactors. After treating the prior scanner, compiler, coefficient, geometry, and object-language interfaces as available, four distinct mechanisms survive the exclusion audit. The first is a reusable semantic theorem; the other three simplify specific maintained proofs.

ID	Change and target	Evidence and cost
P1	Transfer integer polynomial identities to natural split semantics; target algebraic row proofs in the object layer.	Core and literal selection-row client checked. New shared lemmas with an up-front proof cost; no measured project-wide saving.
P2	View the existing sparse-list interpreter as a monomial sum; target `SyntaxCoefficients`.	Complete original and candidate modules checked. 120 to 110 physical lines; 97 to 87 nonblank lines.
P3	Replace manual environment extension by restriction surjectivity; target `sat_noZeroForm_iff`.	Generic mechanism checked. Production declaration proposed at 20 to 8 lines; full object-model integration uncompiled.
P4	Prove the residual exponent is zero from an order obstruction; target `width_dvd_exponent`.	Exact old and new declarations checked, with identical signatures and axiom sets. 12 to 8 lines. Optional prime-radix API checked separately.

For immediate adoption, begin with P2: its entire touched module was checked against the freshly rebuilt current representation module. P4 is also small, but its enclosing module and clients still need an integration build. P3 is a reviewable proof replacement whose final object-language elaboration is explicitly outstanding. P1 deserves a pilot across several algebraic rows before deciding how much shared infrastructure to retain.

The local changes sum to 26 *proposed* removed lines across P2–P4. That total mixes validation scopes: ten lines come from a complete checked module, four from checked exact declarations, and twelve from a proposed production integration. P1 adds new lemmas, whose cost must not be hidden inside that total. There is no claim that the packet makes the repository smaller immediately.

1.1 Source baseline and closing refresh

The initial GitHub main branch was `89226e3921bce3e8949fadd12d3efb6914c7630c`. All 653 blobs in its maintained Lean subtree were matched to the GitHub tree using Git blob SHA-1. The detailed inspection and candidate construction use that snapshot.

A bounded final comparison through `7df43cdfacb490cd3ca3acd23e39027efebf8801` found eleven added Lean modules, aggregate imports, research reviews, and one more incoming ZIP. None of the existing target modules changed. The added report was read and included in the exclusion audit. The comparison receipt is bundled.

This is a snapshot review, not a claim about every later state of a moving branch. The patches identify their exact original bytes.

2 What has already been proposed

The exclusion baseline is the union of all proposals in the 23 placed packets a–w and the five ZIPs in the refreshed incoming directory. The audit reads main reports and substantive supplementary notes, including optional designs, rather than only adoption records. The incoming README was used for routing and relevant searches; unrelated historical manuscripts transitively linked from it were not all reread.

The following families receive no new credit here:

- Optional-list lookup, exact decoder graphs, scanner iteration, validated histories, Boolean reflection, and rule-schedule interfaces.
- Assignment naturality, witness-block layouts, finite residual sums, generic degree and resource bounds, circuit generation, and topological evaluation invariants.
- Coefficient envelopes, finite sparse reflection, coefficient fibres, row operations, and the earlier semiring gate interpreters.
- Canonical carries, cyclic packing, aligned-cell subgroups, helical periods and coordinates, fixed Pell anchors, and coherent word windows.
- Internal definable induction, unique-existence and graph fusion, shared model bridges, internal order constructors, and sentence transport.

2.1 Nearest-precedent comparison

ID	Closest excluded mechanism	Distinct remaining obligation
P1	Standard-natural specialization of semiring gates; reuse of internal evaluators.	Reflect integer-polynomial equality to a natural cross-sum identity and preserve signed zero semantics in arbitrary cancellative commutative semirings.
P2	Finite row coefficient tables and sparse row reflection.	Interpret the existing executable duplicate-preserving list expansion via one sum theorem, replacing several list inductions.
P3	Arity-polymorphic closed-sentence transport.	Simplify the environment-extension proof inside the semantic equivalence consumed by those wrappers.
P4	Coherent source/window APIs assuming recovered powers.	Simplify the exact exponent-remainder proof; optionally recover a block power directly from arithmetic divisibility certificates.

Several apparent opportunities were rejected. Root-constructor minima for `AnnotatedClosure.formCode_ge` are already in the proof-refinements ZIP. Periodic-orbit and lattice descriptions of helices occur in several packets. Signed-prefix borrowing occurs in the last additional interfaces report. That last report also contains the exact `RatioReparameterization.combined_dvd_iff` simplification first considered during this session; its candidate code and claimed savings are omitted.

The final ledger contains 115 inherited contract rows and 61 later overlapping commitments, with 68 report or section-file records across 28 packets. These are audit records, not 176 independent mathe-

matical ideas. The appropriate novelty claim is that no matching mechanism was found in this explicitly audited corpus, not global originality.

3 P1: transport polynomial identities into internal arithmetic

3.1 The proof obligation left by a semiring evaluator

The maintained development has three relevant layers. `Representation: Expr` stores integer arithmetic syntax and its ordinary integer polynomial. `NaturalExpression: splitNatural` translates a signed expression into two natural-coefficient expressions. `GatesSemiring: SemiringZero` interprets vanishing as equality of their natural values.

If e has split (e^+, e^-), write

$$\mathrm{SZ}_R(e, v) \quad :\Longleftrightarrow \quad [\![e^+]\!]_R(v) = [\![e^-]\!]_R(v).$$

Subtraction need not exist on R. An existing semiring evaluator does not automatically permit replacing e by another expression f with the same integer polynomial: their positive and negative syntax pieces can look quite different. Repeatedly unfolding `splitNatural` exposes this implementation to every client.

The proposed interface is

$$\boxed{e.\mathrm{toPolynomial} = f.\mathrm{toPolynomial} \implies (\mathrm{SZ}_R(e, v) \leftrightarrow \mathrm{SZ}_R(f, v)).}$$

It holds for every additively cancellative commutative semiring R. A proof can establish a finite algebraic normalization in an ordinary integer polynomial ring and then reuse it on any appropriate internal-natural carrier. The stored expression itself remains unchanged.

3.2 Proof by a natural cross-sum identity

Let $P_e, N_e, P_f, N_f \in \mathbb{N}[\mathbf{X}]$ be the natural-coefficient polynomials of the four split expressions. Let

$$\iota : \mathbb{N}[\mathbf{X}] \longrightarrow \mathbb{Z}[\mathbf{X}]$$

be the homomorphism induced by the natural-to-integer coefficient map.

One structural induction proves

$$\iota(P_e) - \iota(N_e) = e.\mathrm{toPolynomial}. \tag{1}$$

The constant case is the integer identity $a.\mathrm{toNat} - (-a).\mathrm{toNat} = a$. For multiplication, the positive split is $P_eP_f + N_eN_f$ and the negative split is $P_eN_f + N_eP_f$, so the invariant is the product-of-differences identity. No semantic model occurs in this induction.

If e and f have equal integer polynomials, (1) gives

$$\iota(P_e) - \iota(N_e) = \iota(P_f) - \iota(N_f),$$

hence

$$\iota(P_e + N_f) = \iota(P_f + N_e).$$

Injectivity of the coefficient map, and therefore its multivariate-polynomial map, reflects this to

$$P_e + N_f = P_f + N_e \quad \text{in } \mathbb{N}[\mathbf{X}]. \tag{2}$$

Evaluate (2) through the homomorphism induced by `Nat.castRingHom R`:

$$p_e + n_f = p_f + n_e. \tag{3}$$

If $p_e = n_e$, substitute and cancel a common additive term to obtain $p_f = n_f$. The converse is symmetric.

The proof does not embed the semantic carrier in a ring or construct its Grothendieck completion. It translates a finite coefficient identity once and retains the original semiring semantics.

3.3 Checked Lean interface

The archive's `PolynomialZeroTransport.lean` proves four facts about the actual repository syntax:

```
map_splitNatural_sub
splitNatural_cross_sum
splitNatural_eval_cross_sum
splitNatural_zero_iff_of_polynomial_eq
```

Its main theorem is:

```
theorem splitNatural_zero_iff_of_polynomial_eq
    [CommSemiring R] [IsCancelAdd R]
    (e f : Expr σ)
    (h : e.toPolynomial = f.toPolynomial) (v : σ → R) :
    (e.splitNatural.1.evalSemiring v =
       e.splitNatural.2.evalSemiring v) ↔
    (f.splitNatural.1.evalSemiring v =
       f.splitNatural.2.evalSemiring v)
```

A production wrapper using the existing `SemiringZero` name is definitionally the same statement. The core belongs below the gate and object-model layers to avoid an import cycle. Its checked file imports `NaturalExpressionSemiring`, not the deep object-model closure.

3.4 Concrete client: the conditional selection row

The literal row in `ObjectConditional: literal_selection` is

$$e = o - (bz + (1 - b)p).$$

The object-level goal is the balanced equality

$$o + bp = bz + p. \tag{4}$$

Normalize to

$$f = (o + bp) - (bz + p).$$

The equality of their ordinary integer polynomials follows by `ring`. P1 transfers the zero predicate from e to f; the latter has immediate split semantics (4).

The checked client leaves all four variable indices abstract and then instantiates the exact slots used by `ObjectConditional`: output 0, bit 4, zero-branch 2, and positive-branch 3, with an empty register block. This checks the actual row shape, not a numerical analogue.

This row already has a short direct proof. It demonstrates the interface, not a net saving by itself. The useful adoption experiment is a group of algebraically normalized rows in the zero-test, conditional, pairing, and division syntax layers. Purely structural gates can retain their current cheap specialized lemmas.

3.5 The hypotheses are substantive

Additive cancellation cannot be removed. In the Boolean semiring, with addition OR and multiplication AND, consider

$$e = 1, \qquad f = 2 - 1.$$

Their integer polynomials are both 1. But SZ(e) asks $1 = 0$, which is false, while SZ(f) asks $2 = 1$, which is true in this semiring. The supplied client file constructs the Boolean semiring and checks both evaluations. This is an explicit negative control on the theorem's scope.

Commutative multiplication also matters. Equality in a commutative polynomial ring cannot justify $xy = yx$ in a noncommutative matrix semiring.

Conversely, P1 requires no order, characteristic-zero hypothesis, absence of zero divisors, or multiplicative cancellation. The intended internal-natural application uses the repository's existing semiring and additive-cancellation structure. It never identifies a possibly nonstandard model with external `Nat`.

3.6 Adoption cost and limits

The supplied core has 72 physical lines, including comments and three axiom print commands. It is an added interface, whose whole cost belongs in the accounting. Adopt it when several clients can state their desired balanced equation and discharge only a polynomial identity, with less dependence on the recursive split implementation.

Retain the literal expression, polynomial, witness counts, degree certificates, and all object-model assumptions. The checked core and client prove the semantic mechanism; they do not certify a full rebuild of the internal ZFC proof chain. No project-wide line saving is claimed for P1.

4 P2: give sparse-list expansion one monomial-sum view

4.1 The repeated reasoning

`SyntaxCoefficients`: the interpretation lemmas keeps sparse expansions as executable lists. A term consists of an exponent vector and an integer coefficient. Multiplication distributes through `flatMap` and `map`, and duplicate monomials remain in the list.

The recursive `interpret` function denotes the corresponding ordinary polynomial sum. The original proof layer independently inducts over lists for append, coefficient negation, multiplication by one term, full multiplication, and coefficient extraction. These are all algebraic facts about list sums.

Keep the representation and every executable definition. Add just one private view:

$$\text{interpret}(L) = \sum_{t \in L} \text{monomial}(\text{exponent}(t_1), t_2), \tag{5}$$

where the sum is an occurrence-sensitive list sum, not a finite set sum.

4.2 The new proof structure

Prove (5) once by list induction. Then use the following interfaces:

- Append: `List.map_append` and `List.sum_append`.
- Negation: the additive-map law for monomial coefficients and `List.sum_neg`.
- Multiplication: flatten a list of lists, then use `List.map_flatten`, `List.sum_flatten`, and distribution over sums.
- Coefficient extraction: apply the bundled `MvPolynomial.coeffAddMonoidHom` to the list sum.

The complete multiplication proof becomes

```
simp only [interpret_eq_sum, multiply, List.flatMap_def,
  List.map_flatten, List.sum_flatten, List.map_map,
  Function.comp_def, exponent_add,
  ← MvPolynomial.monomial_mul,
  List.sum_map_mul_left, List.sum_map_mul_right]
```

The private left-multiplication induction is no longer needed. The structural induction proving `interpret_expand` remains: it is the correct connection between expression constructors and the established list algebra.

A small companion derives `exponent_inj` from the injectivity of `Finsupp.equivFunOnFinite.symm`. It replaces a local extensionality proof, and is counted within this module change rather than as a separate proposal.

4.3 Why the executable behavior stays exact

Replacing the list by a finite set would discard duplicate occurrences. This refactor does not do that. Empty lists, repeated exponent vectors, negative coefficients, and coefficients that later cancel are still processed by the original list operations.

Even the existing recursive `interpret` definition is retained, although it is used only for mathematical interpretation. Equation (5) is a theorem about it, so its definitional behavior is preserved.

4.4 Measurements and checking scope

The production candidate is 110 physical lines instead of 120, and 87 nonblank lines instead of 97. These counts include the new private theorem and the explicit list-algebra import. Five list inductions become one; the expression induction remains unchanged.

Both complete modules compile with Lean 4.32.0, using a freshly rebuilt current `Representation` module. Both public `coefficient_expand` endpoints report

$$\{\texttt{propext}, \texttt{Classical.choice}, \texttt{Quot.sound}\}.$$

There is no downstream-build or speedup claim. The compiler times in the logs are execution receipts from a shared environment, not benchmarks.

The exact patch is `patches/syntax-coefficients.patch`. Complete before and after sources are also included.

5 P3: transport quantification by restriction surjectivity

5.1 The actual environment-extension problem

`ObjectPolynomial: sat_noZeroForm_iff` relates validity of a sealed no-zero formula over environments $\mathbb{N} \to V$ to a predicate over finite tuples $\mathrm{Fin}(n) \to V$. Its original proof constructs an environment by hand: use the tuple below n, and a default object elsewhere. It then proves restriction recovers the tuple and transports guards and equation satisfaction separately.

The content is simply surjectivity of restriction. For an injection $j : I \hookrightarrow J$ and a default $d \in V$, the map

$$\rho : (J \to V) \longrightarrow (I \to V), \qquad \rho(e) = e \circ j$$

is surjective. Thus every predicate P on assignments satisfies

$$(\forall e,\ P(e \circ j)) \quad \Longleftrightarrow \quad (\forall a,\ P(a)). \tag{6}$$

Include the naturalness guard and unsolvability conclusion in P.

5.2 Use the existing extension principle

The checked generic mechanism uses Mathlib's `Function.Injective.surjective_comp_right'` and `Function.Surjective.forall`:

```
theorem forall_assignments
    (coordinates : ι → I)
    (hinj : Function.Injective coordinates)
    (default : V) (P : (ι → V) → Prop) :
    (∀ env : I → V, P (env ∘ coordinates)) ↔
      ∀ assignment, P assignment :=
  (hinj.surjective_comp_right' (fun _ => default)).forall.symm
```

For the actual theorem, the injection is `Fin.val` and the default is `vempty H`. No new production helper is necessary: invoke the library fact after the existing satisfaction equivalences.

```
  refine (seal_valid (mem := mem) _).trans ?_
  change (∀ ee : Nat → V,
    Sat mem ee (inputsNaturalForm (Fin.val : Fin n → Nat)) →
      ¬ Sat mem ee (equationForm e Fin.val)) ↔ NoZeroRel H e
  simp only [inputsNaturalForm_spec H, equationForm_spec H]
  exact
    (Fin.val_injective.surjective_comp_right'
      (fun _ => vempty H)).forall.symm
```

The displayed source is wrapped for the page; the supplied patch contains the exact eight-line declaration.

5.3 Boundary cases and distinctness

Injectivity matters. If two tuple coordinates map to the same environment coordinate, arbitrary distinct tuple values cannot be recovered. The universal assignment conclusion would then be stronger.

The explicit default handles the empty coordinate type without silently assuming a nonempty target type in the abstract theorem. In the model application, the empty set already provides this value. No standardness, arithmetic, well-foundedness, or induction property of V is used.

Earlier sentence-transport proposals consume the no-zero equivalence. P3 instead simplifies the environment extension inside that equivalence, so the benefit remains after those earlier wrappers have been applied.

5.4 Evidence and outstanding integration

The generic assignment theorem, its `Fin.val` specialization, and the guarded-unsolvability specialization compile. Their only audited axiom dependencies are the usual `propext`, `Classical.choice`, and `Quot.sound`.

The production declaration changes from 20 to 8 lines, a proposed twelve-line reduction with no added helper. However, the complete `ObjectPolynomial` module was not compiled: its deep object-model dependency closure was unavailable in the local cache. The production patch is marked *integration pending*. Success of the abstract mechanism is not presented as a complete object-module check.

6 P4: use the order obstruction on a residual exponent

6.1 A smaller proof of the existing theorem

`NativeBinaryScale: width_dvd_exponent` proves

$$2^d - 1 \mid 2^u - 1 \quad \Longrightarrow \quad d \mid u$$

with its retained public assumption $0 < d$. Put $r = u \bmod d$. The existing modular-power identity reduces the divisibility hypothesis to

$$2^r - 1 = 0.$$

The old proof turns this into $2^r = 1$, rewrites 1 as 2^0, and invokes injectivity of exponentiation.

The replacement uses the order obstruction:

$$2^r - 1 = 0 \Longrightarrow 2^r \leq 1.$$

If $r \neq 0$, `Nat.pow_le_one_iff` would force $2 \leq 1$. So $r = 0$, and hence $d \mid u$.

```
have hm := Nat.mod_eq_zero_of_dvd hdiv
rw [Nat.pow_sub_one_mod_pow_sub_one] at hm
apply Nat.dvd_of_mod_eq_zero
by_contra hn
have hsmall :=
  (Nat.pow_le_one_iff hn).mp (Nat.sub_eq_zero_iff_le.mp hm)
omega
```

The public signature is unchanged, including `hd`. The supplied source formats the body in six lines; the page breaks one long line. The declaration shrinks from twelve to eight lines. Both exact declarations compile with identical axiom sets. Their unused-`hd` warning is also identical.

6.2 Optional arithmetic API: recover a block power directly

The independently checked `PrimeRadix.lean` proves

$$\begin{aligned} &p \text{ prime}, \quad 1 < q, \quad q \mid p^R, \quad p^d - 1 \mid q - 1 \\ \Longrightarrow \quad &\exists N > 0, \quad dN \le R, \quad q = (p^d)^N. \end{aligned} \tag{7}$$

Classify a divisor of a prime power as $q = p^u$ with $u \le R$. The generalized exponent-divisibility fact yields $u = dN$, and $q > 1$ forces $N > 0$.

An affine adapter accepts $q = 1 + J(p^d - 1)$, matching `sourceQ`. A source client can then supply its divisibility certificate and recover the cell-count exponent and budget together, rather than exposing an intermediate binary exponent.

The generic core needs no separate $d > 0$ hypothesis: for $d = 0$ the other premises force $q = 1$, contradicting $q > 1$. The public source wrapper can still keep its original parameter.

This optional API is not counted as additional removed lines. Its shared proof cost may exceed its benefit for a single current client. Adopt it if the arithmetic classification becomes a reused boundary.

6.3 Necessary guards

The $q > 1$ premise is required for a positive exponent. Primality is also substantive. For

$$p = 6, \quad d = 2, \quad q = 4096, \quad R = 12,$$

one has $4096 \mid 6^{12}$ and $35 \mid 4095$, but 4096 is not a positive power of 36. Thus (7) cannot be generalized to arbitrary bases larger than one. Exponent divisibility alone needs only $p > 1$; primality is needed to classify divisors of p^R.

The exact declaration checks and the optional general prototype are included. The modified complete `NativeBinaryScale` module and its downstream clients were not rebuilt.

7 Validation, costs, and adoption gates

Artifact	What was checked	What remains
P1 core	Actual syntax, split coefficient identity, injective coefficient map, semiring evaluation, and cancellation. Current lower dependencies rebuilt.	Production wrapper and full internal-model clients.
P1 clients	Literal conditional row, exact input slots, and Boolean noncancellation counterexample.	Complete `ObjectConditional` integration.
P2	Complete original and patched sparse-coefficient modules; current representation dependency rebuilt; endpoint axioms matched.	Downstream clients and project checks.
P3	Generic restriction theorem, finite specialization, guarded predicate.	Complete object-polynomial elaboration.
P4 local	Exact old and new declarations; signature and axiom preservation.	Enclosing module and downstream checks.
P4 optional	Prime-power and block-radix arithmetic statements.	Source adapter and helper-cost comparison.

The checked compiler is Lean 4.32.0, commit `8c9756b28d64dab099da31a4c09229a9e6a2ef35`. The repository lock records Mathlib `81a5d257c8e410db227a6665ed08f64fea08e997`. Current representation and natural-expression dependencies were rebuilt for P1 and P2.

The supplied replay script was run successfully: four current production dependencies, six prototype files, and two complete sparse-coefficient modules give twelve successful compiler invocations. All three production patches also apply to their exact recorded baseline sources; patch application is not a replacement for the outstanding module checks.

Successful compiler receipts are included. The checked candidates contain no `sorry` declarations or new axioms. The usual Lean/Mathlib axioms appear in the semantic and coefficient endpoints; the Boolean evaluation counterexamples themselves have no axiom dependencies.

No timing is treated as a speedup, and no finite sample replaces a universal proof. Source-size figures are physical and nonblank lines, not a measure of semantic complexity.

7.1 Before adoption

Use a disposable complete repository checkout with the pinned toolchain. Check each patch against its recorded baseline bytes, rebuild the affected module and direct clients, and inspect relevant endpoint axioms.

For P1, pilot a small group of algebraic row proofs. Keep integer-polynomial equality an explicit certificate; avoid unrestricted polynomial expansion as a global simplification rule. Preserve the literal expression, witness data, degree claims, and model assumptions.

Retain duplicate-preserving list operations (P2), the coordinate injection and default object (P3), and the public signature and optional classifier assumptions (P4).

8 Archive guide and reproduction

The packet is a focused research deliverable, not a repository mirror:

- `report/`: this PDF and its complete editable LaTeX source.
- `lean/`: checked semantic kernels, the exact conditional client, Boolean negative control, and exponent before/after checks.
- `patches/`: production edits for sparse interpretation, environment restriction, and exponent divisibility.
- `baseline/`, `candidates/`: small source snapshots for review.
- `evidence/`: compiler receipts, line counts, source identities, closing GitHub comparison, and exclusion evidence.
- `scripts/`: static archive verification and compiler replay for a complete configured checkout.

Replay does not install a toolchain or silently modify the original repository. The standalone source checks and optional production-patch integration are separate steps. The object-model replacement remains explicitly uncompiled in this delivery.

Primary sources

1. Maintained DiophantineConsistency subtree, closing pin `7df43cdfacb4`.
2. `Representation`: integer syntax and polynomial interpretation; `NaturalExpression`: natural positive/negative syntax; `NaturalExpressionSemiring`: semiring evaluation.
3. `GatesSemiring`: signed zero semantics; `ObjectConditional`: literal selection-row semantics.
4. `SyntaxCoefficients`: executable expansion and coefficient correctness.
5. `ObjectPolynomial`: sealed no-zero semantics on finite internal tuples.
6. `NativeBinaryScale`: binary and block-radix recovery.
7. Placed proposal corpus a–w, including substantive supplementary notes.
8. Five incoming ZIPs, including the final additional interfaces report.
9. Mathlib at the repository lock: function restriction, list sums, polynomial maps, and natural-power arithmetic.